

Machine Learning in Practice: PlantTraits2024

Jasper Pieterse (s1017897) - Rachel Knol (s1010850) - Stefan Boneschanscher (s1011532)

June 7, 2024

1 Introduction

Plant traits describe a plant's function and environmental interactions offering insights into global biodiversity and ecosystem adaptation to climate change. Direct measurement of these traits is impractical on a large scale. Therefore, a deep learning model that extracts traits from images and tabular data could significantly aid in studying plant adaptation in a changing climate [1]. This competition aims to classify six essential plant traits using crowd-sourced images and tabular plant data [2]. The specific goal is to predict the mean values of six specific plant traits: Stem specific density (X4), Leaf area per leaf dry mass (X11), Plant height (X18), Leaf nitrogen content per leaf area (X50), and Leaf area (X3112).

2 Method

Our approach in this competition was to perform some general data preparation found in Section 2.1, and test various architectures which are discussed in Section 2.2 for which we used varying training procedures discussed in Section 2.3. We started by collecting and creating varying baseline models for our different architectures. Using these, we experimented with other data preparation methods such as augmentation, normalization and log-scaling of the labels and features.

2.1 Data Preparation

The dataset includes 55,500 training samples and 6,500 test samples, with images and tabular data. We removed large outliers and negative-valued samples. Data points outside the 0.5th and 98.5th percentiles were excluded. We created a validation set using five stratified splits. We chose this method to ensure even distribution across five intervals.

Image Preparation: Training images were resized to 384x384 pixels. Augmentations included a 50% chance of horizontal flip, a random-sized crop (75% of the time) with a width-to-height ratio of 1.0, resized back to 384x384 pixels and random brightness and contrast shifts (10%) for 25% of images. Images were then normalized using ImageNet mean and standard deviation values and scaled to the $[0,1]$ range.

Tabular Data Preparation: We log-scaled and normalized tabular features to achieve a mean of zero and a standard deviation of one, stabilizing training and improving model performance.

Target Scaling: Targets were normalized using a StandardScaler. During inference, we reversed the normalization to reflect the true data scale.

2.2 Model Architectures

LightGBM Model: We used LightGBM for tabular data, conducting a random search to optimize parameters. We found the parameters to be: 137 estimators, $\eta = 0.019$, maximum depth of 15, 403 leaves, subsample coefficient of 0.770 and the colsample_bytree coefficient to be 0.70.

Regressor Model: The regressor model features an input layer followed by three hidden layers: the first hidden layer has 64 units with batch normalization, ReLU activation, and dropout; the second hidden layer has 32 units with ReLU activation; and the third hidden layer has 20 units with ReLU activation. We used $\eta_{\max} = 1 \times 10^{-4}$, dropout

weight of 0.2, training for 200-1000 epochs depending on whether the CV has converged and the model with the highest validation score was used as the final model.

Image Model: We used a pre-trained Swin Large Vision transformer to classify the 6 plant traits, training for eight epochs with a batch size of 10 and a maximum learning rate of $\eta_{\max} = 1 \times 10^{-4}$.

Image LightGBM Model: Image features were processed separately using the Image model without the final fully connected layer resulting in 1536 image features and stored. The feature images were concatenated with tabular features and the LightGBM model was then trained on the combined features. At inference time the image is first passed to the Image model and the combined features are then passed through the LightGBM model.

Hybrid Model: In the Hybrid model, we combined the Image model and the regressor model. We merged features from two branches into a single feature vector. These get passed through two linear layers with no activation at the end. The model was trained on both the image and tabular data simultaneously.

Ensemble Model: The model parameters of the best-performing LightGBM Model and Image Model are loaded in and their predictions are combined in an ensemble prediction. The weighted contribution of the models to the final submission is 0.2 and 0.8, respectively.

2.3 Training Procedure

Evaluation Metrics The performance of the models in the competition is evaluated by the mean R2 score over all six labels. We tested both R2 and SmoothL1Loss, the latter for its robustness to outliers. It is important to note that R2 assesses errors based on absolute differences rather than relative differences, highlighting the need for careful target transformations (e.g., log-scaling).

Optimizer and Scheduler: For the Deep Learning based architectures, we used the AdamW optimizer with a OneCycleLR scheduler based on validation loss. AdamW was chosen for its efficiency in managing sparse gradients and strong performance in various conditions, such as noisy data and small batch sizes.

Multi-target training: We experimented with multi-target training, predicting both means and standard deviations of plant traits. The latter is referred to as the 'auxiliary task'. Custom loss functions masked NaN values in auxiliary targets. The main task head processes the model's output through a single linear layer without activation to predict mean values. Simultaneously, the auxiliary head also uses a single linear layer without activation to predict standard deviation values.

Test Time Augmentation (TTA): We implemented test time augmentation for the Image model. In test time augmentation predictions are created based on a series of augmentations of the original image. We used twelve different crops of the full-size image. We combined the predictions by computing the mean. The idea to apply this technique on a plant image task is taken from Picek et al [3].

3 Results

Experiment Name	Description	PB	Conclusion
Baseline Model	This Model	0.337	
Loss Log-Scaling	Try disabling log-scaling since it did not work for tabular data. See effect combined with Loss functions.	Off (L1): 0.368 Off (R2): 0.319 On (L1): 0.311 On (R2): 0.337	Refrain from using log-scaling and use L1 Loss.
TTA	Inference over 12 different crops	0.356	0.0113 higher than the same model without.

Table 1: Experiments done with Image model

Experiment Name	Description	PB	Conclusion
Baseline Model	This Model	0.150	Tabular data contains useful information
Feature Normalization	Apply StandardScaler to features	0.150	No effect on PB, slightly decreases training time. Keep implementation.
Target Normalization	StandardScaler applied to targets	0.150	No effect on PB. Abandon idea.
Target Log-Scaling	Log-scaled fat-tailed target distributions (X18, X26, X3112) before normalizing. Reversed log-scaling at inference (after reversing normalization).	0.088	Log-scaling affects PB negatively. Likely because of the absolute nature of R2 Metric. Abandon idea.
Multi-Target Learning	Adding auxiliary features for training and dummy auxiliary features for inference	0.101	Auxiliary features significantly improves training, but does perform worse on the test data.
Feature Log-scaling	Log-scaled all the tabular features.	0.147	Log-scaling features is not useful for tabular data.
Random Search	Did a random search over the hyperparameters	0.153	Best performance yet. Keep these parameters.

Table 2: Experiments done with LightGBM model

Experiment Name	Description	PB	Conclusion
Baseline Model	This Model	0.149	Tabular data contains useful information
Loss Function	Compared L1 Loss to R2 Loss - No Auxiliary Task	R2: 0.150 L1: 0.118	L1 Losses not useful for Tabular Data. Stick to R2 Loss.
Multi-Target Learning	Tried different weights for the auxiliary task head	0.0: 0.150 0.1: 0.144 0.3: 0.141 0.7: 0.139	The auxiliary task worsens scores. Try adding another linear layer to both heads to enable more complex transformations of the feature vector.
Extended heads	Added an extra linear layer to the main and auxiliary head	0.0: 0.145 0.1: 0.142	Extra layers worsen score – Abandon multi-target learning and extra layer.

Table 3: Experiments done with Regressor model

Experiment Name	Description	PB	Conclusion
Hybrid Model	Own Implementation	0.357	No improvement, see discussion.
Ensemble Model	Own implementation with varying ratios for the label weight of both models (tabular-image)	.15-.85: 0.380 .20-.80: 0.380 .30-.70: 0.376	Best model yet. Ratio of 0.20 and 0.80 for tabular and image data respectively gives best performance.

Table 4: Experiments done with the Hybrid & Ensemble model

Experiment Name	Description	PB	Conclusion
Feature extractor v1	Combination of own implementations using an older Image model. For different number of estimators.	150 : 0.3855 300 : 0.3861 600 : 0.3865	Increase in estimators provides improvement in performance, at cost of linear increase in runtime.
Feature extractor v1	increased depth to 14 and no_leaves=2048	150 : 0.3868 1000 : 0.3876	Increases performance, however does not beat the smaller model with 600 estimators.
V1 with new Image model	Using the old feature extractor with a better Image model	600 : 0.366	Decreased performance with new Image model
Feature extractor v2	Same as new Image model, but some bug fixes	150 : 0.374	Better performance but still decreased.
Feature extractor v2	Removed max depth	150 : 0.374	Almost identical performance.

Table 5: LightGBM with image features

4 Discussion

The results from Table 1 suggest that the log-scaling the baseline notebook used, actually decreases performance. Interestingly, using L1 loss appears to improve performance, possibly by providing better guidance through the optimization landscape. Test time augmentation provided a slight increase in performance at the cost of a slower performance during inference time. It could provide a slight edge in the competition, however, we have not experimented with adding it to the more complex models. An additional improvement could be pre-training the model on a more extensive and varied plant image dataset, potentially increasing the model’s ability to detect plant traits.

The results from Table 2 show several findings. Firstly, log-scaling the targets negatively impacts the model’s performance when learning from tabular features, implying that this preprocessing step is unsuitable given the R2 Metric. This suggests a consistent trend where log-scaling is not beneficial. Additionally, other preprocessing steps, such as feature normalization, do not significantly affect the model’s performance. The most notable improvement was achieved through a random search over hyperparameters. Future improvements could focus on creating hand-crafted features based on domain knowledge of the tabular data.

The main takeaway of Table 3 is that the multi-target learning, notably suggested by the competition host themselves, drastically decreases performance. The performance decrease could mean that the multi-target learning approach introduces complexity that the model cannot handle well. Notably, the PB of this regressor model is close to that of the LightGBM model, suggesting that the model type used for the tabular data is of less importance. Notably, the PB achieved by this model is very close to the PB achieved by the LightGBM model.

The results from Table 4 show no improvement compared to the Image model. The lack of significant improvement is likely because we did not use pre-trained image and regressor models. These models train on different timescales; the Image model trains in 6 epochs, while the regressor model requires about 200 epochs. Effective training of the hybrid model in a few epochs would require pre-training, potentially leading to the desired synergistic effect that could outperform an ensemble model. This realization came too late to implement, but it remains an interesting direction.

The feature extraction models, whose results can be found in Table 5, give us some of the highest scores we were able to get. This method appears successful in creating a model that can learn from both the provided image and additional environmental data. While parameter tuning leads to some performance increases, it does not significantly impact overall performance. Contrary to our expectations, increasing the model size did not lead to substantial performance improvements or overfitting. Further experiments using the better-performing Image model with L1 loss were less successful than those with the previous Image model. This may result from log-scaling working better for this specific feature extraction model or from another scaling issue within the feature extraction or training models.

5 Conclusion

It remains challenging to accurately predict plant traits solely based on images and feature data as shown by the relatively low R^2 values. In our study, we discovered that an Image model on its own performs relatively well, but can be slightly improved by incorporating tabular data. Other experiments have shown that log-scaling negatively impacts performance across both tabular and Image models, while hyperparameter tuning and appropriate loss function selection improve results.

Our best working methods were the image feature extraction model and the tabular image ensemble model. Both of these models use the image and tabular data together to increase performance showing the importance of using the tabular data.

Appendices

A Source Code URLs

The accounts “Ringooo,” “Myrthe Kouwenhoven,” and “marijn_klueen” are shadow accounts used to increase submissions and GPU time, resulting in a total of 53 submissions.

- [LightGBM Model training notebook](#)
- [Regressor Model training notebook](#)
- [Image Model training notebook](#)
- [Image Model inference notebook](#)
- [Feature extraction notebook v1](#)
- [Feature extraction notebook v2](#)
- [LightGBM with image features notebook](#)
- [Image Model inference notebook using TTA](#)
- [Hybrid Model inference notebook](#)
- [Ensemble Model inference notebook](#)

B Author Contributions

- Jasper drafted the report, created the Image Model, Regressor Model and LightGBM Model and ran experiments on these.
- Rachel assisted on the report, drafted the presentation, implemented a Image Model based on EfficientNet, implemented a Hybrid Model based on EfficientNet and a Vision Transformer, ran experiments with the Image Model, the LightGBM Model and implemented the Ensemble model.
- Stefan assisted on the report and presentation, implemented the feature extractor and expanded the LightGBM Model to accept features, implemented TTA and ran experiments on these models and the Image Model.

C Evaluation of the Process

In our previous evaluation, we mentioned that we wanted to

1. Spread the workload more equally throughout the weeks.
2. Use a more thorough and methodical approach in streamlining and accelerating our pipeline; to think possible improvements to our model fully through beforehand.
3. Start with a small baseline model and then optimize that pipeline for faster experimentation; build from the ground up even if this means implementing more stuff ourselves.
4. Discuss more with the TAs.

Regarding workload distribution, we mostly succeeded. We completed the Image model pipeline within two weeks, allowing for a more relaxed pace later. However, consistent work distribution throughout the project could have further optimized efficiency.

We didn’t fully achieve our goal of a more methodical approach. Specifically, we implemented log-scaling labels without thoroughly testing its impact. A more rigorous “test-everything” approach, even for seemingly obvious improvements, would have identified this error earlier.

We started with a smaller baseline model, but it could have been even more basic. Including multi-target learning from the beginning proved to be unnecessary work that didn’t improve results. A simpler initial model would have helped as well with troubleshooting the log-scaling issue. It was a textbook example of the “more work is not necessarily better” we have been warned about.

We actively used the Q&A sessions, which helped us identify errors like the log-scaling issue and understand the limitations of multi-target learning despite our shortcomings earlier mentioned. In a way, the Q&A’s kept us on track despite not doing everything perfectly. In the future, we could use sessions like this more effectively by discussing implementation plans before diving in, potentially saving time and effort on unproductive approaches.

D Evaluation of the Supervision

As mentioned, we valued the supervision we received. When we had questions, we got answers and we felt taken seriously. There were also plenty of opportunities to receive feedback.

References

- [1] S. Wolf, M. D. Mahecha, F. M. Sabatini, *et al.*, “Citizen science plant observations encode global trait patterns,” *Nature Ecology & Evolution*, vol. 6, no. 12, pp. 1850–1859, 2022.
- [2] Awsaf, AyushiSharma, HCL-Jevster, inversion, M. Görner, and T. Kattenborn, *Planttraits2024 - fgvc11*, 2024. [Online]. Available: <https://kaggle.com/competitions/planttraits2024>.
- [3] L. Picek, M. Šulc, Y. Patel, and J. Matas, “Plant recognition by ai: Deep neural nets, transformers, and knn in deep embeddings,” *Frontiers in plant science*, vol. 13, p. 787 527, 2022.